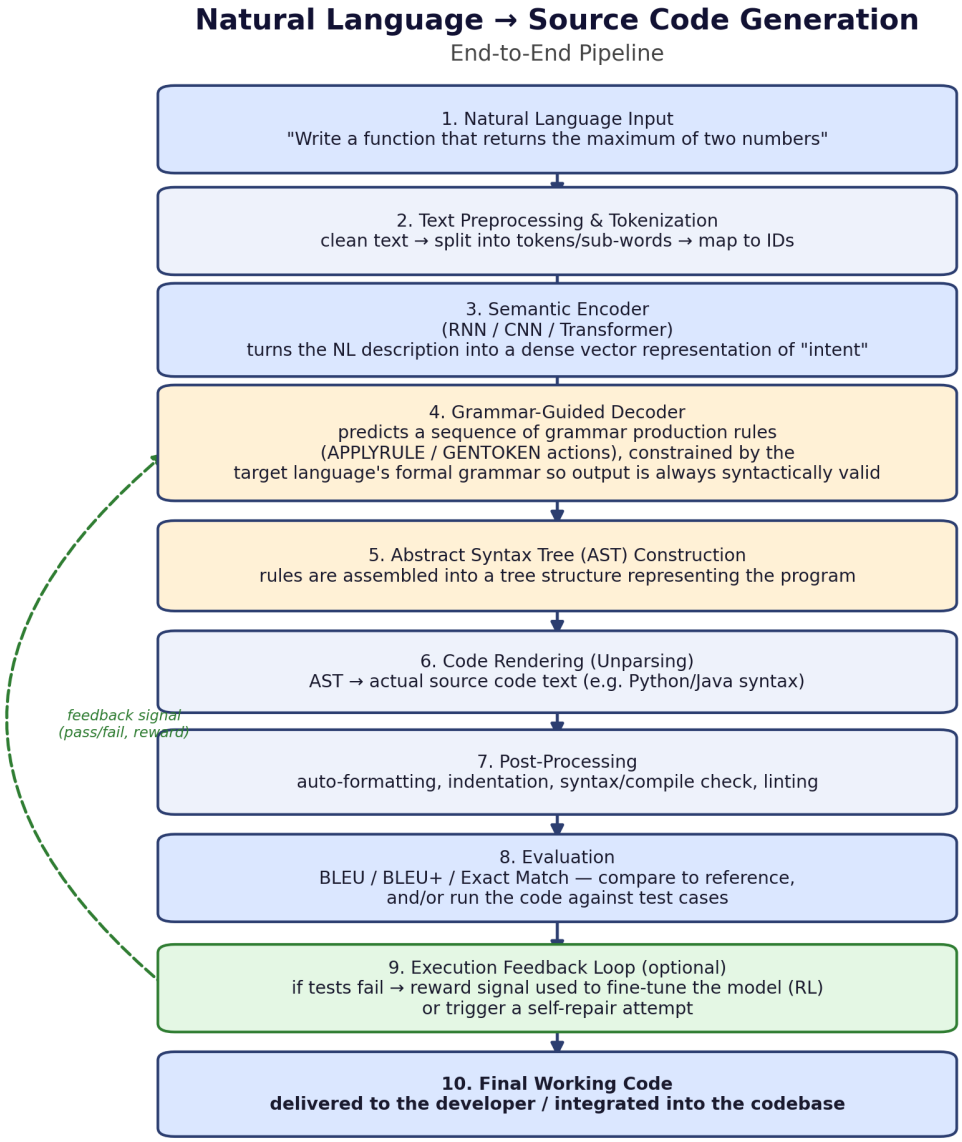
Natural Language → Source Code Generation

Full End-to-End Pipeline: Diagrams, Stage-by-Stage Explanation, and Extensions

---

This document lays out the complete workflow that takes a plain-English description of what a program should do and turns it into working, tested source code — the same general pipeline that the base paper (Alshehri et al., 2025) and most modern code-generation systems follow. It includes the core pipeline diagram, a plain explanation of each stage, and a second diagram showing extra components that can be added on top to make the system more capable, safer, and more accurate.

Diagram 1 — Core End-to-End Pipeline



The straight-line flow from a natural-language request to final, working code, including the optional feedback loop.

Stage-by-Stage Explanation

1. Natural Language Input

The starting point: a plain-English (or any human-language) description of the desired behavior — a docstring, a comment, a ticket description, or a typed prompt. Example: "write a function that returns the maximum of two numbers."

2. Text Preprocessing & Tokenization

The raw sentence is cleaned up and broken into smaller pieces ("tokens" or sub-words) that the model can process numerically. Each token is mapped to an ID number the model understands.

3. Semantic Encoder

A neural network (RNN, CNN, or Transformer) reads the tokenized text and compresses its meaning into a dense numeric vector — essentially the model's internal understanding of "what the user wants." This vector is what every later stage conditions on.

4. Grammar-Guided Decoder

Instead of generating raw text character by character, the decoder predicts a sequence of grammar production rules for the target programming language (e.g., "this is a function definition," "this is a return statement"). Constraining generation to valid grammar rules guarantees the output can never be syntactically broken code.

5. Abstract Syntax Tree (AST) Construction

The predicted rules are assembled into a tree — the same internal structure a compiler builds when it parses code. Each node represents a language construct (loop, condition, expression, variable, etc.).

6. Code Rendering (Unparsing)

The AST is converted ("unparsed") back into actual source code text in the target language's syntax — this is the first point at which the output looks like normal, readable code.

7. Post-Processing

Automatic formatting, indentation, and a syntax/compile check clean up the output and catch anything structurally broken before it reaches a human or a test suite.

8. Evaluation

The generated code is scored, either by comparing it to a reference solution (BLEU, BLEU+, Exact Match) or, more rigorously, by actually running it against real test cases to check functional correctness.

9. Execution Feedback Loop (optional)

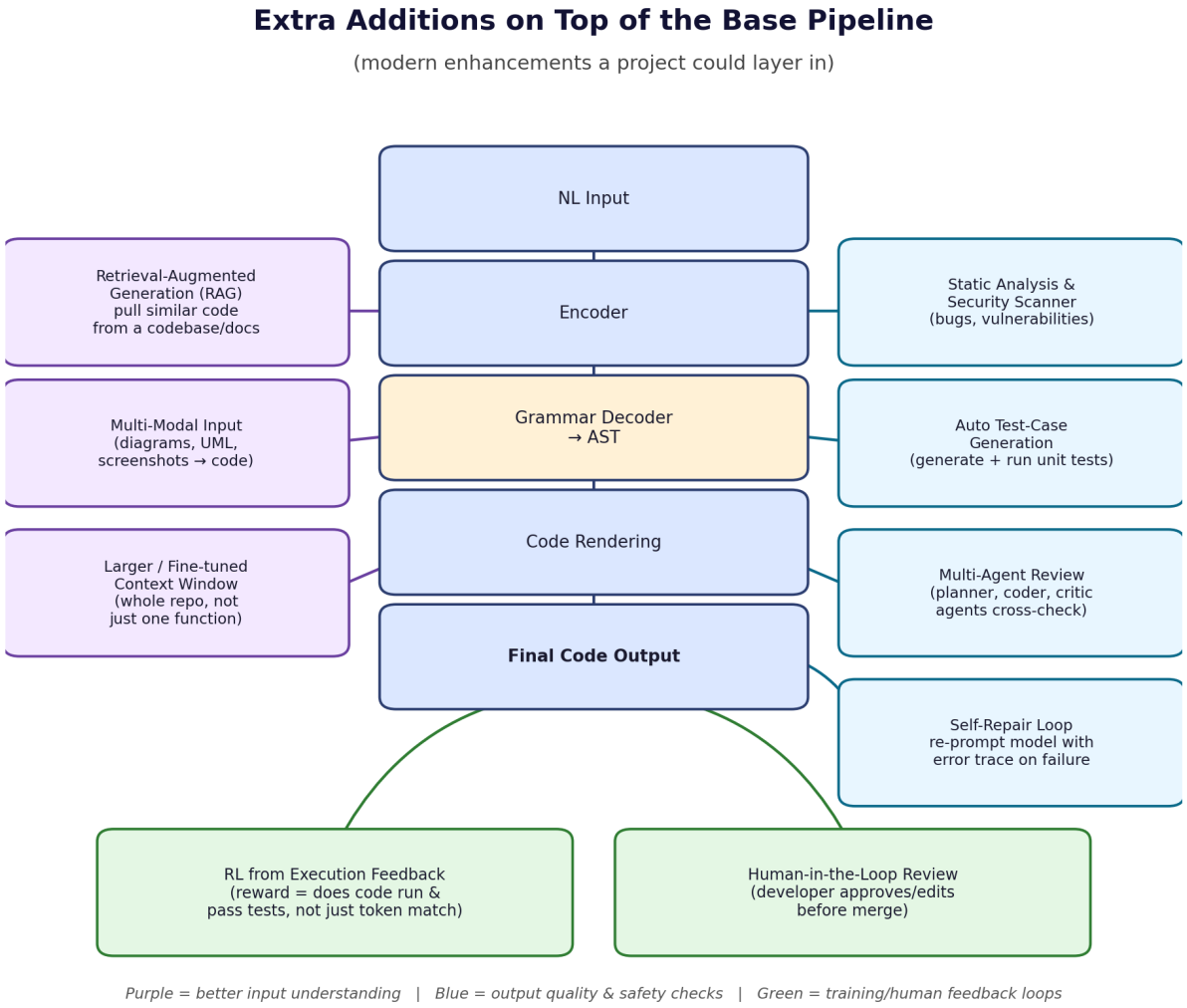
If the code fails its tests, that failure can be turned into a training signal: either used to fine-tune the model via reinforcement learning (reward = did it pass), or fed back into the model as an error message so it can attempt a self-repair.

10. Final Working Code

The finished, validated code is handed back to the developer or merged directly into a codebase.

Diagram 2 — Extra Additions on Top of the Base Pipeline

The core pipeline above is the minimum needed to go from text to code. Real production systems and current research typically bolt on extra components at three points: before the encoder (better input understanding), after code rendering (output quality and safety checks), and around training (feedback loops). The diagram below groups these by color.



Purple = input-side additions, Blue = output-side quality/safety checks, Green = training-time or human feedback loops.

What Extra Can Be Done — Explained

A. Better Input Understanding (Purple)

Retrieval-Augmented Generation (RAG)

Before generating, the system searches a codebase, documentation, or Stack Overflow-style corpus for similar solved examples and feeds the best matches into the encoder alongside the user's request. This grounds generation in real, working code rather than relying purely on what the model memorized during training.

Multi-Modal Input

Accepting more than plain text — UML diagrams, architecture sketches, screenshots of a UI, or even a hand-drawn flowchart — and converting those into the same kind of structured intent the encoder produces from text. Relevant for the base paper's computer-vision domain, where a diagram of a vision pipeline could itself become part of the input.

Larger / Fine-Tuned Context Window

Extending the model to see an entire file or repository, not just an isolated function description, so generated code correctly reuses existing variables, classes, and conventions already in the project.

B. Output Quality & Safety Checks (Blue)

Static Analysis & Security Scanner

Running the generated code through standard static-analysis and vulnerability-scanning tools before it reaches a human, catching issues like SQL injection risks, unused variables, or unreachable code that pure syntax checking would miss.

Automatic Test-Case Generation

Having the system generate its own unit tests for the function it just wrote (or use existing tests from the project) and execute them automatically, giving a functional-correctness signal beyond BLEU-style text similarity — the same motivation behind the base paper's BLEU+ metric, taken further.

Multi-Agent Review

Splitting the task across specialized model "agents" — one that plans the approach, one that writes the code, and one that critiques or reviews it — which tends to catch mistakes a single pass would miss.

Self-Repair Loop

When generated code fails a test or a compile check, the error message is fed straight back into the model as additional context, and the model tries again — closing the loop between stage 8 and stage 4 without needing a full retraining cycle.

C. Training-Time & Human Feedback (Green)

Reinforcement Learning from Execution Feedback

Instead of training the model only to copy a reference answer token-for-token, reward it based on whether its generated code actually compiles and passes tests. This is the direction most 2024–2025 code models have moved toward, as covered in the RL-for-software-engineering literature.

Human-in-the-Loop Review

A lightweight approval step where a developer reviews, edits, or rejects generated code before it is merged — important for any real-world deployment, and a natural place to collect new training data for future fine-tuning.

Where the Base Paper Sits on This Map

> The base paper (Alshehri et al., 2025) implements the core pipeline (stages 1–8) using a CNN encoder, a grammar-guided CNN decoder over an AST, and its own BLEU+ metric for stage 8. It does **not** currently include: retrieval augmentation, multi-modal diagram input, static analysis/security scanning, multi-agent review, an execution-based RL feedback loop, or a self-repair loop. Each of those is a legitimate, well-motivated direction to propose as "future work" or a possible extension of the base paper.

Quick Reference: Extension Ideas and Their Impact

Addition	Solves	Effort to add
RAG / retrieval	Model relies less on memorized patterns	Medium
Multi-modal input	Handles diagrams, not just text	High
Static analysis / security scan	Catches unsafe or buggy code	Low–Medium
Auto test generation + execution	Measures real correctness, not text similarity	Medium
Multi-agent review	Catches mistakes a single pass misses	Medium–High
Self-repair loop	Fixes failures automatically without retraining	Low–Medium
RL from execution feedback	Aligns training with actual correctness	High
Human-in-the-loop review	Safety net before code ships	Low

Summary: the core pipeline (Diagram 1) is what makes natural-language-to-code generation work at all. The extras (Diagram 2) are what separate a research prototype from a production-grade tool — they add grounding, safety, self-correction, and better alignment between what gets rewarded during training and what actually matters: code that runs correctly.